

# An Extra Boost to Boosted Models

Matthew Yang

August 9, 2026

## Contents

|          |                                    |          |
|----------|------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                | <b>1</b> |
| <b>2</b> | <b>Background and Related Work</b> | <b>2</b> |
| <b>3</b> | <b>Aims and Research Questions</b> | <b>2</b> |
| <b>4</b> | <b>Methodology</b>                 | <b>3</b> |
| 4.1      | Replication . . . . .              | 3        |
| 4.2      | Implementation . . . . .           | 3        |
| 4.3      | Benchmarking . . . . .             | 3        |
| <b>5</b> | <b>Timeline</b>                    | <b>3</b> |

## 1 Introduction

Gradient boosted models are built from a sequence of decision trees, where each tree is fitted to the residual errors of the ensemble constructed so far rather than to the original targets [1]. This stage-wise additive formulation, together with highly engineered implementations such as XGBoost [2], LightGBM [3] and CatBoost [4], has made boosting the default choice for tabular prediction tasks, where it remains competitive with, and frequently superior to, deep learning [5, 6].

Boosted models are, however, computationally expensive to train. Inducing a single tree requires evaluating candidate split points over the training set at every node, so the cost of each boosting iteration scales with both the number of instances and the number of features. Approximate split finding, histogram binning, gradient-based sampling and feature bundling all reduce

this cost substantially [2, 3, 7], but they lower the constant factor rather than remove the underlying dependence on the full training set.

Training is also largely sequential. The sequential dependency is intrinsic to boosting itself, since the targets of iteration  $t$  are defined by the predictions of iterations  $1, \dots, t-1$ ; it is compounded by the greedy, top-down induction of each individual tree, in which a node cannot be split until its parent has been. Existing implementations therefore parallelise *within* a boosting iteration, over features or histogram bins, rather than across iterations. This stands in contrast to ensembles such as random forests [8], whose members are independent and thus embarrassingly parallel, but which forgo the error-correcting behaviour that gives boosting its accuracy.

With the ever-growing need to train models on larger datasets, more computationally efficient alternatives to tree-based boosting are needed.

## 2 Background and Related Work

Boosting originates in the observation that a set of weak learners can be combined into an arbitrarily strong one, formalised by AdaBoost [9] and later generalised to arbitrary differentiable losses as gradient boosting [1]. Decision trees are the conventional weak learner, but nothing in the gradient boosting framework requires them: any learner capable of fitting the negative gradient of the loss can be used.

This observation is particularly relevant to time series classification, where random-feature methods have shown that the expensive, data-dependent search performed by tree induction can often be replaced by cheap, data-independent random transformations. ROCKET [10] applies a large bank of random convolutional kernels followed by a linear classifier, achieving accuracy comparable to far more expensive ensembles such as HIVE-COTE [11] at a small fraction of the training cost. MiniRocket [12] and Hydra [13] refine this approach further. The underlying principle of approximating a costly learned representation with randomised features is the same one that motivates random feature approximations to kernel machines [14], and it is the principle we intend to carry into the boosting setting.

## 3 Aims and Research Questions

This research aims to investigate methods of partitioning data without an explicit decision tree, whilst still maintaining the additive, error-correcting structure of a boosted model. We focus particularly on partitioning tech-

niques that can be evaluated in parallel, and we will benchmark these against current methods on both efficiency and accuracy.

Specifically, we ask:

1. Can the traditional decision tree used as the weak learner in gradient boosting be replaced by some form of parallel partitioning algorithm?
2. What is the resulting effect on training time, and how does this approach scale with the number of training instances?
3. What accuracy, if any, is sacrificed relative to state-of-the-art boosted models on standard benchmarks?

## 4 Methodology

This section briefly outlines how we will investigate our proposed research.

### 4.1 Replication

We first attempt to replicate the structure of current state-of-the-art models, in order to establish a baseline against which our own results can be compared.

### 4.2 Implementation

We will mimic the structure of current state-of-the-art models, replacing the tree-based weak learner with a parallel partitioning algorithm and making further changes where necessary.

### 4.3 Benchmarking

The UCR Time Series Classification Archive [15] will provide a quantitative comparison of our models' performance against existing methods, using the archive's default train/test splits so that our results are directly comparable to published ones [16]. We will report both classification accuracy and training time. Accuracy across multiple datasets will be compared using the Friedman test with post-hoc pairwise comparisons, as recommended by Demšar [17], and summarised with critical difference diagrams.

## 5 Timeline

| Week:                           | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---------------------------------|---|---|---|---|---|---|---|---|---|----|----|----|
| Initial meeting                 |   | • |   |   |   |   |   |   |   |    |    |    |
| Review of literature            | • | • | • |   |   |   |   |   |   |    |    |    |
| Writing proposal                |   | • | • |   |   |   |   |   |   |    |    |    |
| Replication of current research |   |   |   | • |   |   |   |   |   |    |    |    |
| Implementation of proposal      |   |   |   |   | • | • | • | • | • | •  |    |    |
| Benchmarking models             |   |   |   |   |   | • | • | • | • | •  | •  |    |
| Project presentation            |   |   |   |   |   |   |   |   |   |    |    | •  |
| Final report                    |   |   | • | • | • | • | • | • | • | •  | •  | •  |

Table 1: Project schedule by week.

## References

- [1] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.
- [4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.
- [5] R. Shwartz-Ziv and A. Armon, “Tabular data: Deep learning is not all you need,” *Information Fusion*, vol. 81, pp. 84–90, 2022.
- [6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?,” in *Advances in Neural Information Processing Systems 35: Datasets and Benchmarks Track*, 2022.
- [7] J. H. Friedman, “Stochastic gradient boosting,” *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.

- [8] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [10] A. Dempster, F. Petitjean, and G. I. Webb, “ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels,” *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.
- [11] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, “HIVE-COTE 2.0: A new meta ensemble for time series classification,” *Machine Learning*, vol. 110, no. 11, pp. 3211–3243, 2021.
- [12] A. Dempster, D. F. Schmidt, and G. I. Webb, “MiniRocket: A very fast (almost) deterministic transform for time series classification,” in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, (New York), pp. 248–257, ACM, 2021.
- [13] A. Dempster, D. F. Schmidt, and G. I. Webb, “Hydra: Competing convolutional kernels for fast and accurate time series classification,” *Data Mining and Knowledge Discovery*, vol. 37, pp. 1779–1805, 2023.
- [14] A. Rahimi and B. Recht, “Random features for large-scale kernel machines,” in *Advances in Neural Information Processing Systems 20*, pp. 1177–1184, 2007.
- [15] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, “The UCR time series archive,” *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.
- [16] M. Middlehurst, P. Schäfer, and A. Bagnall, “Bake off redux: A review and experimental evaluation of recent time series classification algorithms,” *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, 2024.
- [17] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.